UNIVERSITÉ DU QUÉBEC

UQTR Université du Québec à Trois-Rivières

GÉGI

GEI1092-00

Techniques d'intelligence artificielle

Mini-projet 1

par

Sébastien Cabana

CABS 7203 0202

Étudiant en génie informatique

sebastien.cabana@uqtr.ca

Yohan Lefebvre

LEFY84100106

Étudiant en génie informatique

yohan.lefebvre@uqtr.ca

26 février 2026

UQTR ÉCOLE D'INGÉNIERIE

UNIVERSITÉ DU QUÉBEC

UQTR Université du Québec à Trois-Rivières

GÉGI

Plan de présentation

- Création et entrainement du modèle
- Évaluation et optimisation du modèle
- Analyse de l’empreinte écologique
- Considération éthique
- Implémentation sur système embarqué
- Conclusion

UQTR ÉCOLE D’INGÉNIERIE

Préparation des données

```python
# Define the folders to process
folders = [
    './aclImdb/test/neg',
    './aclImdb/test/pos',
    './aclImdb/train/neg',
    './aclImdb/train/pos'
]
```

```python
# Process each folder and collect data
data = []
for folder in folders:                                          # Iterate through each folder in the list
    if os.path.exists(folder):                                  # Check if the folder exists
        sentiment = 0 if 'neg' in folder else 1                 # Determine sentiment based on folder name
        for filename in os.listdir(folder):                     # Iterate through each file in the folder
            if filename.endswith('.txt'):
                file_path = os.path.join(folder, filename)      # Construct the full file path
                try:
                    with open(file_path, 'r', encoding='utf-8') as file:   # Open the file with UTF-8 encoding
                        review_text = file.read().strip()       # Read the review text and remove leading/trailing whitespace
                    data.append({'review': review_text, 'sentiment': sentiment})   # Append the review and sentiment to the data list
                except Exception as e:
                    print(f"Error reading {file_path}: {e}")    # Print an error message if there is an issue reading the file
```

```python
# Create dataframe from collected data
dataset = pd.DataFrame(data, columns=['review', 'sentiment'])
```

```
.
├── LICENSE
├── Mini_Project1.ipynb
├── README.md
├── aclImdb
│   ├── README
│   ├── imdb.vocab
│   ├── imdbEr.txt
│   ├── test
│   │   ├── labeledBow.feat
│   │   ├── neg
│   │   ├── pos
│   │   └── urls_neg.txt
│   └── train
│       ├── labeledBow.feat
│       ├── neg
│       ├── pos
│       ├── unsup
│       ├── unsupBow.feat
│       ├── urls_neg.txt
│       ├── urls_pos.txt
│       └── urls_unsup.txt
└── emissions.csv
```

<id>_<rating>.txt

UQTR ÉCOLE D'INGÉNIERIE

Prétraitement du texte

```python
# Remove HTML tags
dataset['review'] = dataset['review'].apply(lambda x: re.sub('<.*?>', '', x))

# Remove special characters (keep only letters and spaces)
dataset['review'] = dataset['review'].apply(lambda x: re.sub(r'[^a-zA-Z\s]', '', x))

# Convert to lowercase
dataset['review'] = dataset['review'].apply(lambda x: x.lower())

# Remove stop words and perform stemming
stop_words = set(stopwords.words('english'))
ps = PorterStemmer()
def crop_review(text):
    words = text.split()                                                    # Split the text into individual words
    filtered_words = [ps.stem(word) for word in words if word not in stop_words]    # Remove stop words and apply stemming
    return ' '.join(filtered_words)                                         # Join the processed words back into a single string
dataset['review'] = dataset['review'].apply(crop_review)                    # Apply the crop_review function to the 'review' column
```

Source : https://www.linkedin.com/pulse/understanding-tf-idf-rany-elhousieny-phd%E1%B4%AC%E1%B4%AE%E1%B4%B0/

Extraction de critères

```python
# Initialize TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer(ngram_range=(1, 2),  # Use unigrams and bigrams
                                   min_df=5,            # Ignore words that appear in less than 5 documents
                                   max_df=0.8)          # Ignore words that appear in more than 80% of documents

# Split features and target
X = dataset['review']
y = dataset['sentiment']

# Apply TF-IDF vectorization
X_tfidf = tfidf_vectorizer.fit_transform(X)

# Get feature names
feature_names = tfidf_vectorizer.get_feature_names_out()

print(f"Original text shape: {X.shape}")
print(f"TF-IDF matrix shape: {X_tfidf.shape}")
print(f"Number of features: {len(feature_names)}")
```

✓ 15.4s

```
Original text shape: (50000,)
TF-IDF matrix shape: (50000, 181459)
Number of features: 181459
```

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

```python
# Convert text data into numerical features using TF-IDF
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(df['message'])
```

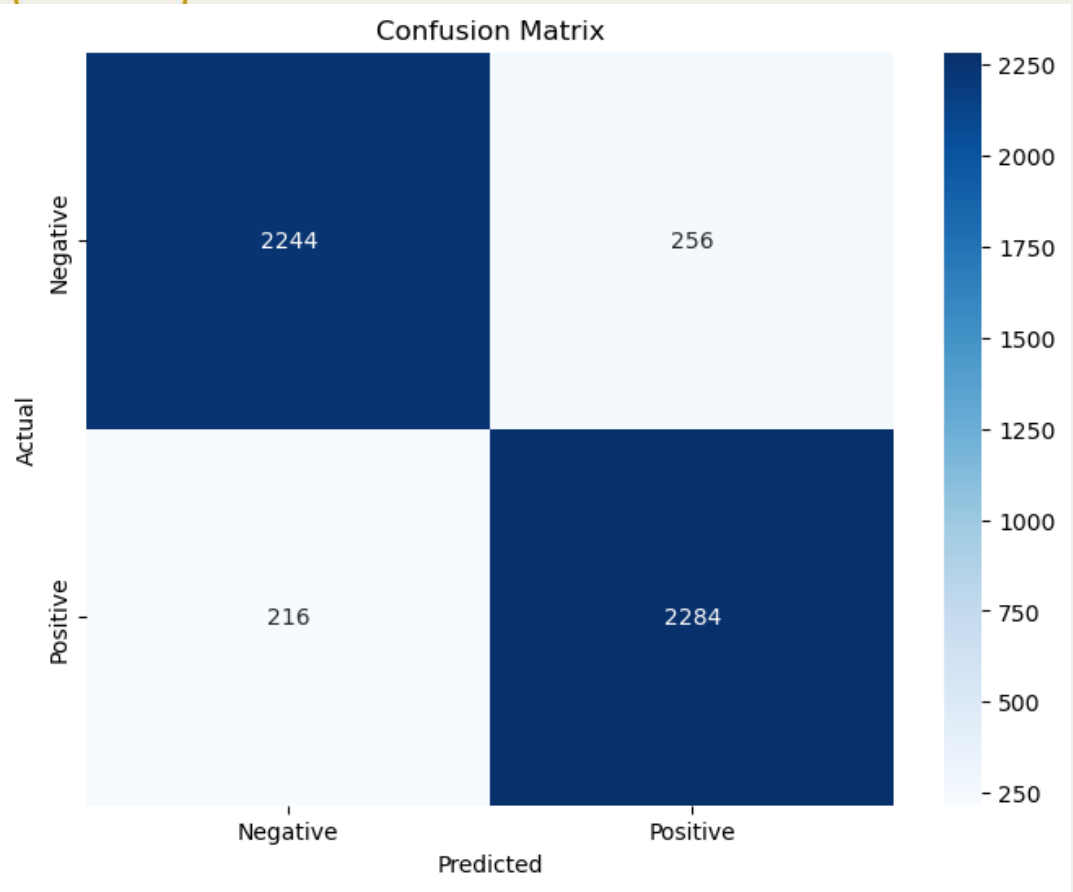

UQTR ÉCOLE D'INGÉNIERIE

Entraînement du modèle

```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_tfidf, y, test_size=0.1, random_state=1, stratify=y)

# Initialize the Logistic Regression model
lr = LogisticRegression(C=10.0, random_state=1, solver='lbfgs', multi_class='ovr')

# Train the model
lr.fit(X_train, y_train)
```

Évaluation du modèle

```
Classification Report:
              precision    recall  f1-score   support

Negative (0)       0.91      0.90      0.90      2500
Positive (1)       0.90      0.91      0.91      2500

    accuracy                           0.91      5000
   macro avg       0.91      0.91      0.91      5000
weighted avg       0.91      0.91      0.91      5000
```



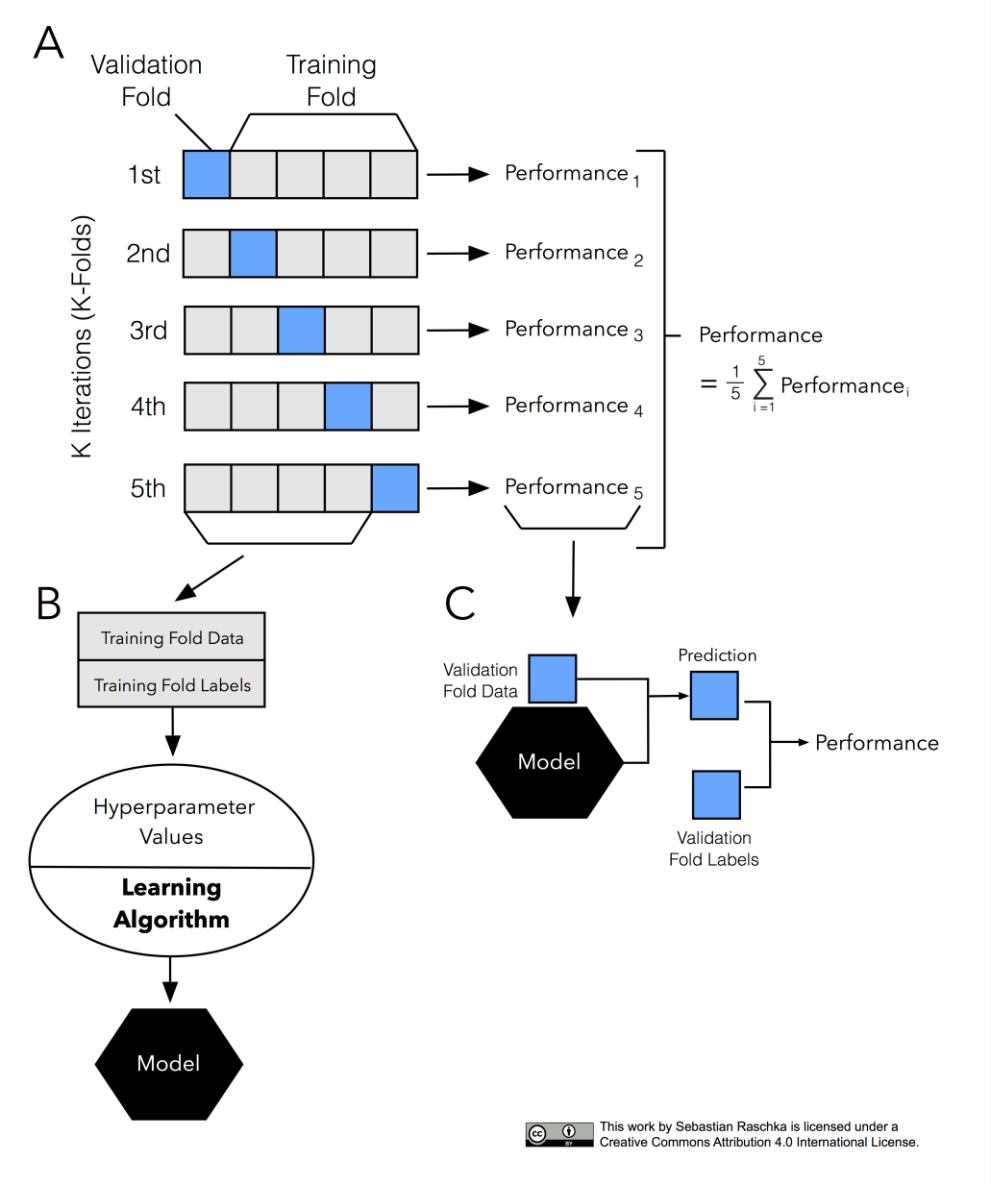

```python
# Make predictions
y_pred = lr.predict(X_test)

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Negative (0)', 'Positive (1)']))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
```

Source : https://stackoverflow.com/questions/58391295/how-is-cross-validation-performed-and-how-gridsearchcv-specifically

Ajustement des hyperparamètres

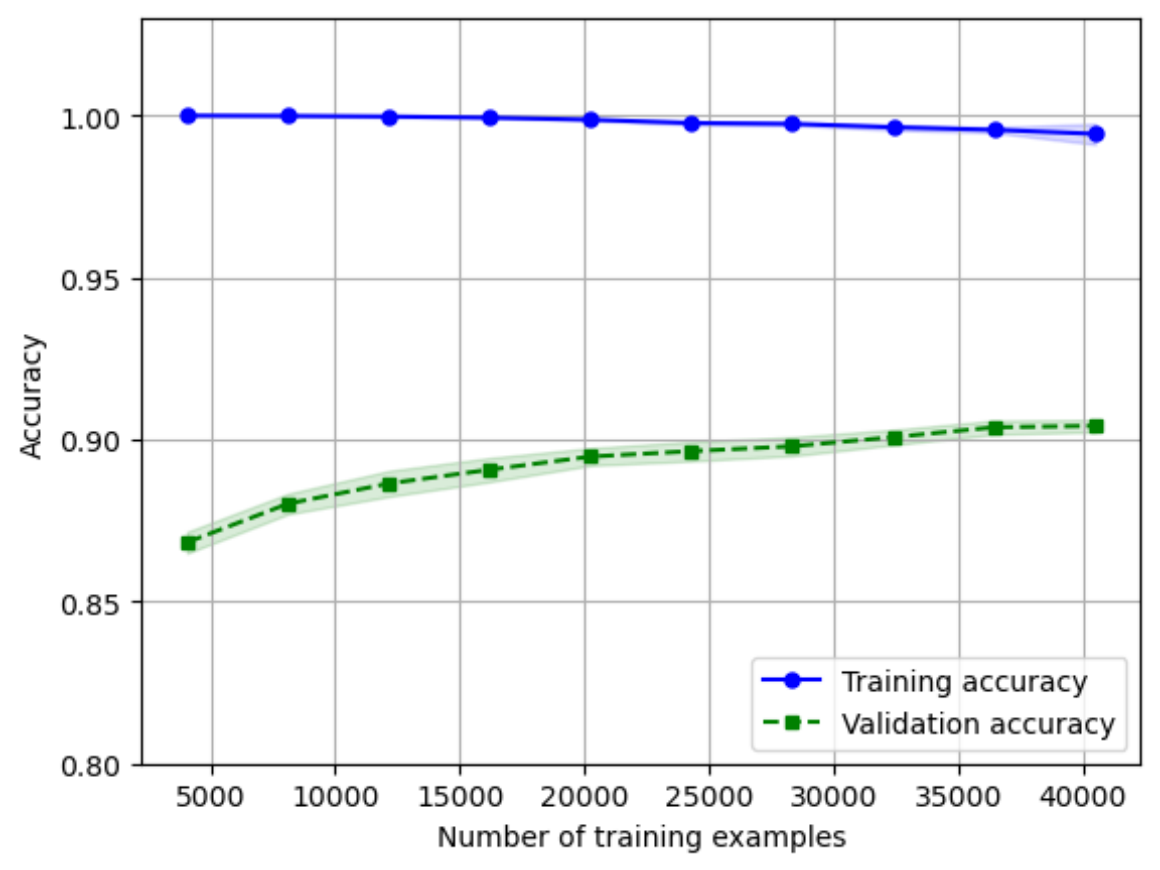
```python
param_range = [0.0001, 0.001, 0.01, 0.1, 1.0, 10.0, 100.0, 1000.0]
gs = GridSearchCV(estimator=lr,
                  param_grid={'C': param_range},
                  scoring='accuracy',
                  cv=10,
                  n_jobs=-1)

# Perform Grid Search to find the best C parameter
gs.fit(X_train, y_train)

# Print the best C parameter found by GridSearchCV
print(f"Best C parameter: {gs.best_params_['C']}")
```

```
Best C parameter: 10.0
```



UQTR ÉCOLE D'INGÉNIERIE

Analyse de la courbe d'apprentissage




```python
train_sizes, train_scores, test_scores =\
    learning_curve(estimator=lr,
                   X=X_train,
                   y=y_train,
                   train_sizes=np.linspace(0.1, 1.0, 10),
                   cv=10,
                   n_jobs=1)
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)

plt.plot(train_sizes, train_mean,
         color='blue', marker='o',
         markersize=5, label='Training accuracy')

plt.fill_between(train_sizes,
                 train_mean + train_std,
                 train_mean - train_std,
                 alpha=0.15, color='blue')

plt.plot(train_sizes, test_mean,
         color='green', linestyle='--',
         marker='s', markersize=5,
         label='Validation accuracy')

plt.fill_between(train_sizes,
                 test_mean + test_std,
                 test_mean - test_std,
                 alpha=0.15, color='green')

plt.grid()
plt.xlabel('Number of training examples')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.ylim([0.8, 1.03])
plt.show()
```

Empreinte écologique

```python
# TASK 8: Track emissions during model training
tracker = EmissionsTracker()
tracker.start()

# Train the model
lr.fit(X_train, y_train)

# Stop the emissions tracker and save the results
emissions = tracker.stop()
print(f"CO2 emissions during training: {emissions} kg")
```

```
Energy consumed for all CPU : 0.000033 kWh
Energy consumed for all GPUs : 0.000005 kWh
CO2 emissions during training: 1.74e-07 kg
```

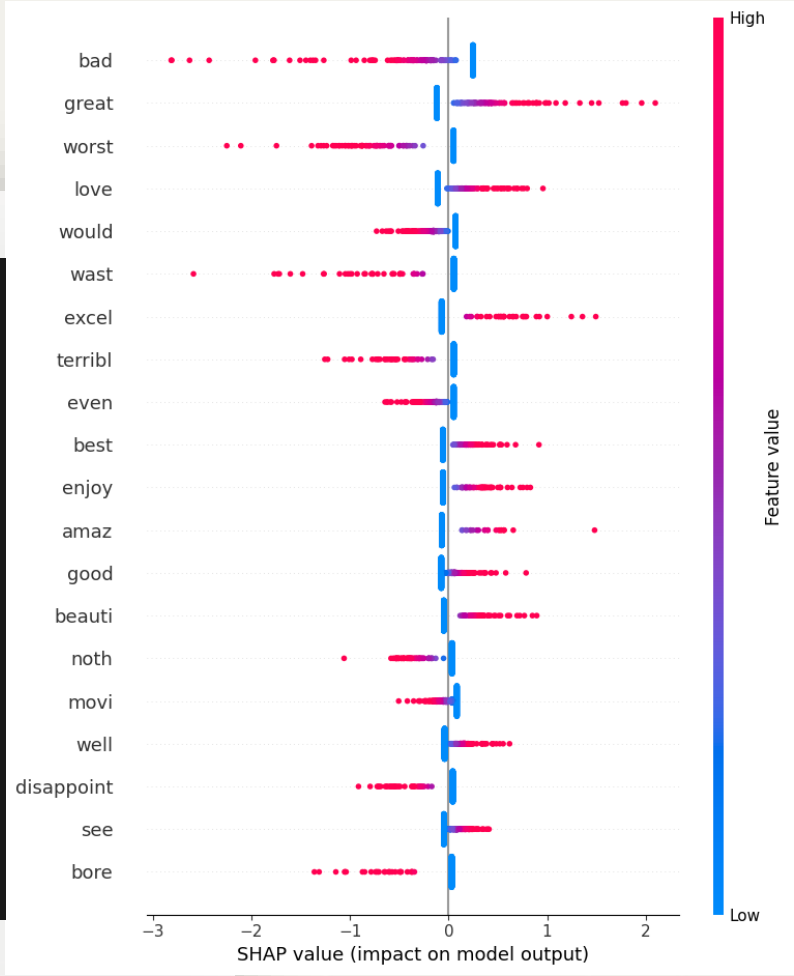
```python
# TASK 8: Track emissions during Hyperparameter Tuning
tracker_hp = EmissionsTracker()
tracker_hp.start()

# Perform Grid Search to find the best C parameter
gs.fit(X_train, y_train)

# Stop the emissions tracker and save the results
emissions = tracker_hp.stop()
print(f"CO2 emissions during training: {emissions} kg")
```

```
Energy consumed for all CPU : 0.000538 kWh
Energy consumed for all GPUs : 0.000022 kWh
CO2 emissions during training: 2.58e-06 kg
```

Considérations éthiques

- SHAP
- Biais dans les données
- Structure de phrase
- Sarcasme
- Contrôle des données
- Analyse des résultats

```python
# Show SHAP summary plot with proper feature names
shap.initjs()

# Échantillon train
np.random.seed(1)
bg_size = 500 #Recommandation entre 100-1000 choix de 500 arbitraire
bg_idx = np.random.choice(X_train.shape[0], bg_size, replace=False)
X_bg = X_train[bg_idx]

# Création explainer
explainer = shap.LinearExplainer(lr, X_bg, feature_names=tfidf_vectorizer.get_feature_names_out())

# Échantillon test
sample_size = 500
sample_idx = np.random.choice(X_test.shape[0], sample_size, replace=False)
X_sample = X_test[sample_idx]
X_sample_dense = X_sample.toarray()

# Calcul SHAP
shap_values = explainer.shap_values(X_sample)

# Affichage
shap.summary_plot(
    shap_values,
    X_sample_dense,
    feature_names=tfidf_vectorizer.get_feature_names_out(),
    show=True
)
```



UQTR ÉCOLE D'INGÉNIERIE

Déploiement sur système embarqué

- Exportation des données
- Échantillons des TOP features
- Conversion
- Génération du fichier

```python
# Réglage
TOP_K = 1000                    # Nb de features conservées
W_BITS = 16                     # poids en int16 pour réduire mémoire requise
X_FRAC = 12                     # 2^x_frac: utiliser pour transformer les float en int sans perte de précision
W_MAX = 2**(W_BITS - 1) - 1     # Valeur max pour scaling
THRESHOLD_Q = 0                 # Utiliser pour hard decision sur arduino
MODEL_H_PATH = "sentiment_model.h"
```

```python
# Export des données du modèle
w = lr.coef_.ravel()
b = float(lr.intercept_[0])
feature_names = tfidf_vectorizer.get_feature_names_out()

# Sample des features les plus importants
top_idx = np.argsort(np.abs(w))[-TOP_K:][::-1]
w_k = w[top_idx]
names_k = feature_names[top_idx]

#Scaling
max_abs_w = np.max(np.abs(w_k))
W_SCALE = W_MAX / max_abs_w
w_q = np.round(w_k * W_SCALE).astype(np.int16)
b_q = int(np.round(b * W_SCALE * (2**X_FRAC)))
THRESHOLD_Q = 0
```

```c
// Auto-generated quantized logistic regression model for Arduino
// Generated from TF-IDF + LogisticRegression (Top-K reduction + fixed-point quantiza

#include <stdint.h>

#define MODEL_NUM_FEATURES 1000
#define MODEL_X_FRAC 12

// Quantized weights (int16)
static const int16_t MODEL_W[MODEL_NUM_FEATURES] = { -32767, -28259, -26058, -23249,

// Quantized bias (int32)
static const int32_t MODEL_B = 281795;

// Decision threshold (logit >= 0 => positive)
static const int32_t MODEL_THRESHOLD = 0;

// Top-K feature names
static const char* MODEL_FEATURE_NAMES[MODEL_NUM_FEATURES] = { "worst", "aw", "wast",

// Quantization scale factors
static const float MODEL_W_SCALE = 1684.7327206741f;
// x_q = round(x_float * 2^MODEL_X_FRAC)

// Fixed-point logit computation
static inline int32_t model_logit_q(const int16_t* x_q) {
  int64_t acc = (int64_t)MODEL_B;
  for (int i = 0; i < MODEL_NUM_FEATURES; ++i) {…
  }
  if (acc > INT32_MAX) return INT32_MAX;
  if (acc < INT32_MIN) return INT32_MIN;
  return (int32_t)acc;
}

// Final prediction function
static inline int predict_sentiment(const int16_t* x_q) {
  return (model_logit_q(x_q) > MODEL_THRESHOLD) ? 1 : 0;
}
```

Des questions ?